

File permissions in Linux

Project description

The research team at my organization requires updates to the permissions for specific files and directories in the projects directory. At present, the permissions do not match the required authorization levels. Verifying and adjusting these permissions helps maintain system security. To accomplish this, I executed the following steps:

Check file and directory details

I ran the `ls -la` command to list all directory contents, including hidden files. The resulting output reveals five project files, one hidden file named `.project_x.txt`, and a directory called `drafts`. In this list, the 10-character string located in the first column defines the current access permissions for each item.

```
researcher2@bf0be37ccdbe:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Aug 24 16:29 .
drwxr-xr-x 3 researcher2 research_team 4096 Aug 24 16:29 ..
-rw--w---- 1 researcher2 research_team  46 Aug 24 16:29 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Aug 24 16:29 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Aug 24 16:29 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Aug 24 16:29 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 24 16:29 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 24 16:29 project_t.txt
researcher2@bf0be37ccdbe:~/projects$
```

Describe the permissions string

In Linux, file and directory permissions are represented by a 10-character string. The permissions themselves are defined as:

- **Read (r):** For files, this allows reading contents; for directories, this grants the ability to read all contents, including files and subdirectories.
- **Write (w):** For files, this allows modifying content; for directories, this grants the ability to create new files.

- **Execute (x):** For files, this allows executing the file as a program; for directories, this allows entering the directory to access files.

The 10-character string structure is broken down as follows:

- **1st character:** This character is either a 'd' or a hyphen (-) and indicates the file type. A 'd' signifies a directory, while a hyphen (-) indicates a regular file.
- **2nd-4th characters:** These indicate the read (r), write (w), and execute (x) permissions for the user. A hyphen (-) here indicates that the permission is not granted.
- **5th-7th characters:** These indicate the read (r), write (w), and execute (x) permissions for the group. A hyphen (-) here indicates that the permission is not granted for the group.
- **8th-10th characters:** These indicate the read (r), write (w), and execute (x) permissions for others (all other users on the system apart from the user and the group). A hyphen (-) here indicates that the permission is not granted for others.

Change file permissions

The first two lines of the screenshot display the commands I entered, and the other lines display the output of the second command. The `chmod` command changes the permissions on files and directories. The first argument indicates what permissions should be changed, and the second argument specifies the file or directory. In this example, I removed write permissions

from other for the `project_k.txt` file. After this, I used `ls -la` to review the updates I Made.

```
researcher2@bf0be37ccdbe:~/projects$ chmod o-w project_k.txt
researcher2@bf0be37ccdbe:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Aug 24 16:29 .
drwxr-xr-x 3 researcher2 research_team 4096 Aug 24 16:29 ..
-rw--w---- 1 researcher2 research_team  46 Aug 24 16:29 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Aug 24 16:29 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Aug 24 16:29 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Aug 24 16:29 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 24 16:29 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 24 16:29 project_t.txt
researcher2@bf0be37ccdbe:~/projects$
```

Change file permissions on a hidden file

The research team at my organization recently archived `project_x.txt`. They do not want anyone to have write access to this project, but the user and group should have read access. The following code demonstrates how I used Linux commands to change the permissions:

```
researcher2@bf0be37ccdbe:~/projects$ chmod u-w,g-w,g+r .project_x.txt
researcher2@bf0be37ccdbe:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Aug 24 16:29 .
drwxr-xr-x 3 researcher2 research_team 4096 Aug 24 16:29 ..
-r--r----- 1 researcher2 research_team  46 Aug 24 16:29 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Aug 24 16:29 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Aug 24 16:29 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Aug 24 16:29 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 24 16:29 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 24 16:29 project_t.txt
researcher2@bf0be37ccdbe:~/projects$
```

Change directory permissions

I executed the command `chmod g-x drafts` to restrict execute permissions for the group on the `drafts` directory, and then verified the change by running `ls -la`. It was previously determined that the group had execute permissions, so I used the `chmod` command to remove them. The `researcher2` user already had execute permissions, so they did not need to be added.

Summary

To align file and directory access within the `projects` directory with organizational authorization standards, I performed a series of permission updates. I began by evaluating the existing permissions using the `ls -la` command, which guided the necessary modifications. Subsequently, I applied the `chmod` command across multiple iterations to adjust access settings for the designated files and directories.